

Rapport de Sécurité et Bonnes Pratiques de l'Application de Gestion de Parc Informatique

Date de génération : 2025

Version de l'Application : 1.0

Contributeurs principaux : JULIANS Jean-Yann et WAHAGA Clément

Objectif : Réécrire et sécuriser une application de gestion de parc informatique initialement réalisée en PHP « pur » pour qu'elle réponde aux exigences de sécurité OWASP Top 10, en respectant une architecture MVC moderne et en appliquant toutes les bonnes pratiques PHP (notamment celles énoncées dans « PHP: The Right Way »).

1. Tableau Récapitulatif des Fonctionnalités de Sécurité

Fonctionnalité	Description	Mesures mises en place
Architecture MVC	Séparation claire entre la logique métier (Modèles), le contrôle de la logique de présentation (Contrôleurs) et l'affichage (Vues).	Organisation du code en dossiers distincts : /app/models/, /app/controllers/, /app/views/ permettant un découplage et une maintenabilité optimale.
Gestion des Sessions	Gestion centralisée des sessions pour contrôler l'accès et authentifier les utilisateurs, avec vérification sur chaque contrôleur.	Appel unique ou conditionnel à session_start() (via if (session_status() == PHP_SESSION_NONE)) afin d'éviter les doublons et warnings. Stockage de \$_SESSION['user_id'] pour l'accès.
Protection CSRF	Prévention des attaques CSRF via l'intégration d'un token généré et validé pour chaque formulaire de l'application.	Les formulaires incluent un champ caché <input type="hidden" name="csrf_token" value="..."> généré par la fonction csrf_token() (définie dans /app/helpers/security.php) et validé via validate_csrf_token().
Sanitization des Entrées	Nettoyage systématique de toutes les entrées utilisateur pour éviter les injections SQL et XSS.	Utilisation de filter_var() avec le filtre FILTER_SANITIZE_FULL_SPECIAL_CHARS pour les chaînes et FILTER_SANITIZE_NUMBER_FLOAT avec FILTER_FLAG_ALLOW_FRACTION pour les nombres dans les contrôleurs.
Utilisation de Requêtes Préparées (PDO)	Toutes les interactions avec la base de données se font via PDO et requêtes préparées afin d'éviter toute injection SQL.	Dans les modèles (Materiel, Peripherique, etc.), toutes les requêtes utilisent PDO avec des bindParam() ou execute() afin de sécuriser les accès à la base de données.
Echappement des Données en Sortie	Sécurisation des données lors de l'affichage dans les vues pour éviter l'exécution	Utilisation d'un helper (fonction e(), définie dans /app/helpers/functions.php) qui encapsule htmlspecialchars(), appliquée partout dans les vues pour sécuriser toutes les sorties dynamiques.

Fonctionnalité	Description	Mesures mises en place
	de code malveillant dans le navigateur (protection XSS).	
Configuration HTTPS via .htaccess	Redirection systématique des requêtes HTTP vers HTTPS afin de sécuriser les communications.	Fichier .htaccess dans le dossier public configuré avec une règle RewriteCond pour rediriger toute requête non HTTPS vers HTTPS.
Configuration Apache et php.ini	Renforcement de la sécurité au niveau du serveur via des directives permettant de masquer les erreurs et de désactiver les fonctions potentiellement dangereuses.	Utilisation de directives dans le fichier .htaccess (ex: php_flag display_errors Off) et ajustement du fichier php.ini pour activer log_errors, définir un chemin d'erreur, et désactiver exec, shell_exec, etc.
Envoi de Headers de Sécurité HTTP	Ajout de headers pour limiter les risques de clickjacking, de content-sniffing et d'autres attaques côté client.	Les headers tels que X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block, Referrer-Policy: no-referrer, et Content-Security-Policy: default-src 'self' sont envoyés dans le front controller.
Support UTF-8 Complet	Utilisation cohérente de l'encodage UTF-8 dans toute l'application pour éviter les problèmes d'affichage et de traitement des chaînes.	Utilisation dans les DSN PDO du charset utf8mb4, configuration de mb_internal_encoding('UTF-8') et mb_http_output('UTF-8') dans un include global, et définition des métadonnées HTML en UTF-8.

2. Description Détaillée des Mesures Apportées

2.1. Architecture MVC

L'application a été structurée selon le modèle MVC (Modèle-Vue-Contrôleur) :

- **Modèles** : Tous les accès à la base de données se font via des classes modèles utilisant PDO et des requêtes préparées (ex. Matériel, Périphérique).
- **Contrôleurs** : Ils gèrent la logique de traitement (ajout, mise à jour, suppression, recherche) et s'assurent que les données transmises sont sanitisées et validées. Ils gèrent également la sécurité via la vérification CSRF et la gestion de session.
- **Vues** : Les données affichées aux utilisateurs sont traitées dans des vues séparées, intégrant des helpers pour l'échappement (fonction e()) et l'affichage des messages.

Cette séparation permet une meilleure maintenabilité, une meilleure réutilisation du code et une isolation de la logique métier, ce qui contribue à une application plus sécurisée et plus robuste.

2.2. Sécurisation de la Session et Contrôle d'Accès

La gestion des sessions est centralisée :

- La session est démarrée dans le front controller ou vérifiée avec `if (session_status() == PHP_SESSION_NONE) { session_start(); }` dans chaque contrôleur.
- Les pages protégées vérifient la présence de `$_SESSION['user_id']`, empêchant ainsi l'accès non autorisé.

2.3. Protection CSRF

Chaque formulaire de l'application intègre un champ caché contenant un token CSRF généré via la fonction `csrf_token()` du helper de sécurité.

- Ce token est stocké dans la session et est validé lors de la soumission (via `validate_csrf_token()`) afin d'empêcher toute soumission frauduleuse de formulaires par un tiers.

2.4. Sanitization des Entrées

Avant le traitement ou l'insertion en base, toutes les entrées utilisateur sont nettoyées (sanitisées) avec `filter_var()`.

- Pour les champs textuels, on utilise `FILTER_SANITIZE_FULL_SPECIAL_CHARS` afin d'échapper les caractères spéciaux.
- Pour les valeurs numériques, `FILTER_SANITIZE_NUMBER_FLOAT` avec le flag `FILTER_FLAG_ALLOW_FRACTION` est utilisé.

Cela empêche les injections SQL ainsi que les tentatives d'introduction de code malveillant dans l'application.

2.5. Requêtes Préparées via PDO

Toutes les interactions avec la base de données se font via PDO en utilisant des requêtes préparées et des paramètres liés, garantissant ainsi que les données insérées dans les requêtes sont correctement échappées pour éviter les injections SQL.

2.6. Echappement en Sortie

Les données affichées dans les vues sont passées par une fonction helper `e()`, qui utilise `htmlspecialchars()` pour échapper les caractères spéciaux.

- Cela prévient toute possibilité d'exécution de code malveillant injecté dans des contenus dynamiques (protection XSS).

2.7. Configuration Serveur via .htaccess et php.ini

Au niveau du serveur, plusieurs mesures ont été mises en place :

- **Fichier .htaccess dans le dossier public :**
 - Force la redirection de HTTP vers HTTPS.
 - Configure PHP pour masquer les erreurs (`display_errors Off`) et activer la journalisation (`log_errors On`).

- Définit des paramètres de sécurité pour les cookies de session (session.cookie_secure, session.cookie_httponly, session.cookie_samesite Strict).
- **Configuration php.ini de XAMPP :**
 - Désactive l’affichage des erreurs à l’écran en production.
 - Active le log des erreurs dans un fichier défini.
 - Désactive des fonctions PHP potentiellement dangereuses (exec, shell_exec, etc.), afin de réduire la surface d’attaque.

2.8. Envoi de Headers HTTP de Sécurité

Dans le front controller, des headers de sécurité sont envoyés pour renforcer la sécurité côté client :

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- X-XSS-Protection: 1; mode=block
- Referrer-Policy: no-referrer
- Content-Security-Policy: default-src 'self'
- Ces headers aident à protéger contre le clickjacking, le content sniffing et d’autres attaques basées sur les failles du navigateur.

2.9. Support UTF-8

L’application travaille entièrement en UTF-8 pour garantir une manipulation correcte des caractères internationaux.

- Le DSN PDO inclut charset=utf8mb4 afin de garantir le support complet de l’UTF-8.
- Dans les scripts (ou un fichier global), la fonction mb_internal_encoding('UTF-8') est utilisée pour définir l’encodage interne, et mb_http_output('UTF-8') assure que l’output est en UTF-8.

2.10. Utilisation des Helpers

Des helpers personnalisés ont été développés pour :

- **Filtrer et échapper** les données (e()) pour htmlspecialchars, et dans security.php pour le CSRF).
- Ces helpers centralisent la logique de filtrage et rendent le code plus maintenable et sûr.

3. Conclusion

L’application de gestion de parc informatique, initialement réalisée par JULIANS Jean-Yann, a été entièrement revue pour se conformer aux exigences de sécurité OWASP Top 10. Parmi les améliorations notables :

- Adoption d’une architecture MVC permettant une séparation claire entre logique métier et présentation.

- Mise en place d'un système de gestion des sessions et de contrôle d'accès robuste.
- Intégration de tokens CSRF et vérification sécurisée dans tous les formulaires.
- Sanitization systématique des entrées via `filter_var()` avec `FILTER_SANITIZE_FULL_SPECIAL_CHARS` et d'un échappement sûr via une fonction helper `e()`.
- Utilisation de requêtes préparées PDO pour protéger l'accès à la base de données.
- Configuration des headers de sécurité HTTP et des paramètres serveur pour forcer l'utilisation de HTTPS et masquer les erreurs en production.
- Garantir la bonne gestion de l'encodage UTF-8 dans l'ensemble de l'application.
- Désactivation de fonctions dangereuses dans `php.ini` pour limiter la surface d'attaque.

Ces mesures, mises en œuvre au travers d'un code structuré et de la configuration serveur, assurent que l'application est non seulement conforme aux pratiques recommandées par l'OWASP Top 10, mais également prête à être déployée dans un environnement de production sécurisé.